

Error Correction for DNA Storage

Jin Sima, Netanel Raviv, *Senior Member, IEEE*,
Moshe Schwartz, *Senior Member, IEEE*, and Jehoshua Bruck, *Life Fellow, IEEE*

I. INTRODUCTION

Information theory is considered by many as the mathematical theory of *communication*. Normally, the word “communication” describes a scenario involving two physically distant parties that exchange information, but may equally involve two *temporally* distant parties that do so. The latter gives rise to communication across time, rather than across space, and is commonly referred to as *information storage* i.e., the process of encoding information into a physical device in order to retrieve it at a later point in time, efficiently and accurately.

In his groundbreaking 1948 paper, Claude Shannon (1916-2001) showed that *all* types of information (images, text, videos, etc.) can be communicated using bits, i.e., zeros and ones, and an identical statement holds in the case of storage. In order to store a piece of information, one has to encode it using bits, and place those bits on a reliable physical device, preferably a *non-volatile* one, i.e., that does not require electric current to retain that information.

The earliest example of high-density non-volatile storage device (beyond punch-cards and written media which existed for millennia) is probably that of *magnetic storage*. In this 1950’s technology, bits were organized on a magnetizable tape using different magnetization patterns. Over the following decades, increased demand for higher storage volumes pushed this technology forward to become the *hard-disk drives*, which in recent years cleared the way to *solid-state drives*.

Albeit over 10 orders of magnitude increase in volume since they were introduced, digital storage devices struggle to keep up with increasing storage demands.

J. Sima is with the Department of Electrical and Computer Engineering, University of Illinois Urbana-Champaign, Urbana 61801, IL, USA (e-mail: jsima@illinois.edu).

N. Raviv is with the Department of Computer Science and Engineering, McKelvey School of Engineering, Washington University in Saint Louis, St. Louis, MO, 63130 USA (email: netanel.raviv@wustl.edu).

M. Schwartz is with the Department of Electrical and Computer Engineering, McMaster University, Hamilton, ON, Canada (e-mail: schwartz.moshe@mcmaster.ca)

J. Bruck is with the Electrical Engineering Department, California Institute of Technology, Pasadena, CA, 91125 USA (e-mail: bruck@caltech.edu)

The immense volume of data generated today, especially since the emergence of information sharing platforms such as YouTube and social networks, is projected to pass the rate in which digital storage devices improve. Especially prominent is the growing requirement for “cold” storage, i.e., one which is seldom accessed, such as old family photos or historical records. One of the most promising and most radical new technologies to resolve the cold storage problem, is *DNA storage*, i.e., storing information in DNA molecules.

In a way, DNA molecules are the storage device of nature. These long molecules, which contain sequences of either one of four basic molecules called nucleotides A, C, G, T, are used by all living organisms to communicate across time. The DNA molecules contain “recipes” for producing proteins, which are the building blocks of living organisms. By communicating DNA across generations, these recipes are literally transmitted from parents to offsprings, and enable life to continue.

In the past several decades, scientists have had remarkable success in creating artificial DNA molecules in the lab, and in keeping those molecules stable in a vial (i.e., not inside any living cell). The A, C, G, T nucleotides in these artificial molecules can be chosen freely, and can therefore store bits just like any other storage device. For instance, one can decide that

$$A = 00, C = 01, G = 10, T = 11$$

and then store the sequence 00101001 by creating the DNA molecule AGGC in the lab. DNA storage has fantastic and far-reaching advantages over existing technology:

- *DNA storage is ultra-dense.* Current data centers are the size of buildings; storing similar amounts of data in DNA would require the size of a refrigerator.
- *DNA storage is stable.* DNA molecules can last tens of thousands of years without any energy investment; some off-the-shelf hard drives will not be usable in as little as 20 years.
- *DNA storage is future-proof.* As long as there are humans, DNA reading technology will be of interest, and DNA reading devices will exist. This can hardly be said about, say, floppy disks, whose reading nowadays requires a trip to a museum. In

other words, while humans already almost forgot how to read some fairly recent storage devices, **humans will never forget how to read DNA.**

A typical process to store information in DNA is as follows: First, the data, represented by 0's and 1's, are encoded into sequences of nucleotides. Then, the DNA molecules containing these sequences of nucleotides are synthesized and stored either in vials or in living organisms, which is the data writing process. To read the data, the Polymerase Chain Reaction (PCR) technology is used to access the part of the data to be retrieved. Then, the DNA molecules obtained after the PCR process are read using DNA sequencing techniques, thereby recovering the sequences of nucleotides. Finally, the sequences of nucleotides are decoded back to the data. A more detailed description of a DNA storage workflow is given in Section III.

Whether old or new, all storage devices are prone to *errors*. Due to imperfect hardware, physical damage, or deterioration of materials, some bits in any storage device might be read in error. Without proper preparation, losing even a single bit might render the respective piece of information unreadable, and therefore lost. Coding-theorists and engineers have been combating this phenomenon ever since storage devices were invented, and various *error-correction* mechanisms, known as *codes*, were developed.

All these mechanisms require adding *redundancy* to the data, i.e., to store more bits than the actual size of the data. These redundant bits are then used in the reading process in cases where some bits are read in error. The simplest form of redundancy is *replication*: instead of storing every bit of the data once, store it 3 times. For example,

Data:	01001
Store:	000111000000111

Then, while reading a possibly error-filled sequence from the device, the most frequent bit among every consecutive triple is most likely the correct one:

Store:	000111000000111
Errors:	100110000001111
Correction:	0 1 0 0 1

But how much redundancy is enough? The example above shows a three-times increase in the amount of storage, a high price to pay. The minimal amount of redundancy required to correct errors is an ongoing and difficult research area in coding theory, and depends on the type of storage medium. Coding theorists have worked relentlessly over the past 80 years in order

to come up with algorithms that guarantee error-free information storage for the existing storage technology. However, as we shall see throughout this paper, DNA storage devices have a very unique structure and constraints, which give rise to new and interesting **types of errors which have never been studied.**

The error shown above is called *substitution*: a “1” bit is replaced by a “0” bit, or vice versa. This is a common and well-understood error in traditional storage media, which seldom appears in DNA storage devices. However, most errors in DNA storage devices are new, i.e., have never appeared in traditional devices. These new types of errors depend on the type of DNA storage device at hand.

DNA storage devices are partitioned to two families: the most common family is called *in-vitro*, which includes devices that contain a vial with short, unordered sequences of DNA that float in a solution inside that vial. The other less common family is called *in-vivo*, where artificially synthesized DNA molecules are planted inside a primitive life form, such as bacteria, for better data longevity and stability. Better longevity is guaranteed by the self-sustaining property of primitive life-forms; with minimal energy investment a bacteria colony could last millions of years. Better stability is guaranteed by reproduction across generations; redundancy in the data will be introduced naturally, and will be stabilized via natural selection.

In-vitro DNA storage devices pose several interesting coding-theoretic challenges. First, they are prone to known but understudied errors called *deletions*. In a deletion, a bit completely disappears from a sequence without leaving a trace, and the read sequence is shorter than the one which was initially stored. Deletions occur in DNA storage mainly due to imperfections in the synthesis reaction, which commonly skip one or more of the nucleotides that need to be written due to weak chemical bonds. A closely related type of errors, where redundant bits sneak into the sequence unexpectedly, are called *insertions*.

Even though mechanisms for deletion correction, known as deletion codes, were studied to some extent since the 1960's, much was left unknown. Driven by the advent in DNA storage, the interest in deletion errors increased recently, and optimal solutions were found only in the past few years. Deletion errors will be described in Section II.

An even more substantial challenge in in-vitro DNA storage systems is the fact that only *short and unordered* DNA sequences can be stored together in a vial. Current limitation in DNA-synthesis technology can only generate sequences that are a few thousand nucleotides long,

and placing those sequences together in a vial makes it impossible to know which one comes after or before any other. This is in sharp contrast to traditional storage media, where data is partitioned to *pages*, that always appear in memory in the same order they were written.

A simple solution to the ordering problem comes in the form of *indexing*: begin each short DNA sequence with several nucleotides that determine its correct position relative to other sequences in the same vial. Surprisingly, this is *not* the best solution in terms of the amount of redundancy. Due to errors, the indexing nucleotides might get scrambled and interfere with the correct order. An optimal solution for the order problem was also found very recently, and it is described in Section III.

For in-vivo storage, however, the picture is remarkably different. While placing the synthetic DNA inside bacteria improves its longevity and stability, it exposes the stored data to the natural bio-chemical and evolutionary processes inside the bacteria. As the reader might already know, cells reproduce by a process called *mitosis*, where one cell splits into two. During mitosis, DNA molecules replicate themselves in each one of the offspring cells, a process that is not perfect, and some errors might occur. In nature, these errors are the basis of Darwin's natural selection theory: arbitrary errors cause arbitrary mutations, and only the mutations which improve the organism's ability to survive persist among generations. In the context of information storage, however, these errors must be understood, and corrected; an *evolution-correcting code* must be developed. A common error in this setting is *duplication*, where a piece of DNA material is replicated, and attaches itself at a different location. Duplication errors in in-vivo DNA storage are described in Section IV.

II. DELETION CODES

Though deletion errors were studied from the 1960's, motivated by synchronization errors in traditional media, the interest in correcting deletion errors increased recently due to their prevalence in DNA storage. As mentioned in Section I, deletions, insertions, and substitutions, are the notable three types of errors that occur in the reading, writing, and storing processes in DNA storage. Hence, codes correcting these three types of errors are necessary for reliably storing information in DNA. Beyond the applications in DNA storage and communication, the study of deletions, insertions, and substitutions, is also connected to edit distance and sequence alignment, etc., which have applications in natural language processing and other applications involving DNA sequence analysis.

A. What are deletion, insertion, and substitution errors?

We now describe the three types of errors in greater detail. A deletion removes a symbol from a sequence; in the context of DNA storage, it removes a nucleotide from the sequence, e.g., turning TGGA into TGA. In the context of natural language, it removes a letter from a word or a sentence, e.g., turning the word "cat" into "at." An insertion adds an extra symbol to the sequence, e.g., turning ACTG into ACCTG or turning "eat" into "heat." A substitution replaces a symbol in the sequence, e.g., changing TGGA to TGGG or "for" to "far."

Among those three types of errors, substitution errors are better understood compared to deletions and insertions, as there are many classic code constructions for correcting substitution errors such as Hamming codes, polar codes, LDPC codes, Reed-Muller codes, Reed-Solomon codes, and BCH codes, etc. Many of these codes were proved to be optimal in terms of redundancy in some settings. However, less is known about deletion and insertion errors, which are commonly referred to as *synchronization* errors. Nevertheless, the information-theorist Vladimir Levenshtein (1935-2017) proved an interesting fact about deletion and insertion errors already in the 1960's: if a code corrects deletion errors, it can also correct an equal number of combination of deletions and insertions (However, an efficient encoding/decoding algorithm for correcting deletions does not necessarily imply an efficient algorithm for correcting deletion and insertion errors). Moreover, a substitution error can be regarded as a deletion error followed by an insertion. Therefore, it is reasonable to focus on deletion errors, as correcting deletion errors implies correcting a combination of the three types of errors.

There are two scenarios for correcting deletion errors:

- 1) The *probabilistic* scenario, where a fraction of deletions occur randomly. The goal is to find the optimal information rate (i.e., the data size relative to the code length), known as the *channel capacity*, such that the information could be recovered with high probability.
- 2) The *adversarial* scenario, where at most a certain number of deletions are caused by an adversary that wishes the reading process to fail. The goal is to find the minimal amount of added redundancy that guarantees successful reading in all cases.

DNA storage devices are commonly considered as an adversarial scenario, since the number of deletions is usually quite small, and the respective amount of redundancy can be optimized directly. Therefore, in this section we focus on *adversarial deletion correcting codes*. Moreover, for simplicity we consider bits rather

than nucleotides, however, similar statements can be made for nucleotides as well.

In contrast to multiple results about correcting substitution errors, there are not many efficient and well structured codes correcting deletion errors, and even the deletion channel capacity is still unknown except for cases where the deletion probability is small. One of the reasons for deletions and insertions being more difficult is that channels with substitution channels are *memoryless*, i.e., different output bits are independent given the input, while deletion/insertion channels are not; one deletion affects all subsequent bits by shifting them one position to the left. Moreover, there is a symmetry in substitution errors that is not present in deletion or insertion errors.

To see this symmetry, we introduce the notion of an *error ball*, which is common in the analysis of error correcting codes. An error ball is the set of all possible sequences one can get after at most some number of errors occur in a given input sequence. If the type of error is substitution or deletion, we call the corresponding error ball a substitution ball or a deletion ball, respectively.

For example, the substitution ball and the deletion ball of the input sequence 1001 with at most 1 error are given by $\{1001, 0001, 1101, 1011, 1000\}$ and $\{1001, 001, 101, 100\}$, respectively. The symmetry in substitution errors reflects the fact that the size of the substitution ball is independent of the input sequence; any other sequence of length 4 will have a substitution ball with 5 sequences. Moreover, the erroneous sequence is uniformly distributed over the substitution ball if the substitution indices are uniformly and randomly selected. These two properties do not hold for deletion balls, since two deletion balls (for different input sequences) can have different sizes. In addition, the probabilities of getting different erroneous sequences in the deletion ball are different when the deletion indices are uniformly and randomly selected. The following example illustrates this crucial difference in symmetry.

Example 1. Consider the sequences 0000 and 1010. The substitution balls of 0000 and 1010, with at most a single substitution, are $\{0000, 1000, 0100, 0010, 0001\}$ and $\{1010, 0010, 1110, 1000, 1011\}$, respectively. Each set has 5 elements and each element appears once under all possible error patterns. The deletion balls of 0000 and 1011 with at most a single deletion are given by $\{0000, 000\}$ and $\{1011, 011, 111, 101\}$, respectively. The numbers of elements in the two sets are different. In addition, after a single deletion in 1011, the erroneous sequence becomes 011 or 111 if the first bit or the second bit is deleted, respectively. The erroneous sequence be-

comes 101 if either the third or the fourth bit is deleted. Hence, it is more probable to obtain 101 than 011 or 111 after a single deletion uniformly occurs in 1011.

B. How to efficiently correct deletions?

One of the natural approaches to correct deletion errors is to use a *repetition code*, which was presented in the introduction for substitution errors but also works well for deletion errors. This is undesirable due to high redundancy. Another potential approach is to borrow results from substitution correcting codes. The difficulty with this approach is that even two binary sequences with large *Hamming distance* (i.e., have multiple different bits), which are resilient to substitution errors, can be ambiguous even under a single deletion. As an example, one can consider two sequences 1010101 and 0101010 that have Hamming distance 7 (i.e., all bits are different), and therefore one can identify the correct sequence between them under any 3 substitutions. However, these two sequences become indistinguishable in the case of even a single deletion, since deleting the first bit in one produces the same string 010101 as deleting the last bit in the other.

How to correct a single deletion with low redundancy? One classic construction is the Varshamov-Tenengolt (VT) codes, defined by

$$C_0 = \{(c_1, \dots, c_n) : \sum_{i=1}^n ic_i \equiv 0 \pmod{n+1}\}, \quad (1)$$

which corrects a single deletion. In words, VT codes show that one can protect any sequence from a single deletion by summing up the indices of all the 1 entries in the sequence and taking a modulo of $n+1$. The following example demonstrates how this modulo summation scheme works.

Example 2. Suppose the erroneous sequence 00110 is obtained from some unknown sequence $c_1c_2c_3c_4c_5c_6$ after a single deletion. To see what $c_1c_2c_3c_4c_5c_6$ is, enumerate all possible sequences of length 6 that might become 00110 after a single deletion, given by 000110, 001010, 001100, 100110, 010110, 001110, and 001101. The respective modulo summations, i.e., the expressions $\sum_{i=1}^6 ic_i \pmod{n+1}$, are given by 2, 1, 0, 3, 4, and 5, respectively, all of which are different. Therefore, the only sequence with modulo summation 0 is 001100, and hence it must be the correct answer.

The VT code has redundancy at most $\log_2(n+1)$ bits, which is asymptotically optimal for a single deletion.

Can we generalize the VT code to correct more than a single deletion with asymptotically optimal redundancy? This is a key question toward a good understanding

of how to correct deletion errors, and turns out to be highly nontrivial. A counting argument by Vladimir Levenshtein showed that the optimal redundancy of codes of length n correcting t deletions lies between $t \log_2 n + o(\log_2 n)$ and $2t \log_2 n + o(\log_2 n)$ for constant t , where the notation $o(\log_2 n)$ means that $\frac{o(\log_2 n)}{\log_2 n}$ approaches 0 as n goes to infinity. For larger t , the optimal redundancy is linear in $t \log_2(\frac{n}{t})$. These arguments only prove that a code exists, without finding it explicitly. Finding the respective codes explicitly with comparable redundancy has been puzzling for decades even for the case $t = 2$.

Inspired by the modulo summation in VT codes, researchers wondered if *higher-power* summation might be useful for correcting multiple deletions with optimal redundancy. Specifically, given the modulo sums $\sum_{i=1}^n i^p c_i$ for p from 0 up to some positive integer, is it possible to recover $(c_1, \dots, c_n)$ from multiple deletions? This is a challenging question even for two deletions, and unfortunately, counterexamples were found showing that knowing the sums $\sum_{i=1}^n i^p c_i$ for p 's from 0 up to $p = 4$ does not guarantee successful correction.

To generalize the idea of using weighted modulo sum for correcting multiple deletions, one can use weights that are exponential in indices in the weighted modulo sum. However, due to the exponential weights, such generalization requires redundancy that is linear in the code length to correct even two deletions, in contrast to the optimal redundancy which is a logarithm in the code length for correcting a constant number of deletions.

Another brilliant idea for correcting multiple deletions is to use a concatenated code, that has a two-level structure of an inner code and an outer code. Specifically, the codewords are separated into blocks. The inner code protects each block from deletions and is constructed by using exhaustive search (i.e., finding the best code by traversing all codes using a computer). The outer code treats each block as a symbol and uses a substitution correcting code to correct blocks in case the inner code fails in some blocks. Note that the brute force search to construct the inner code is tractable when the block size is small (specifically, a logarithm of the code length). The concatenated code approach reduces the problem of correcting deletions in a long sequence to that of correcting deletions in short sequences, by using the well-constructed substitution correcting codes. Using concatenated codes, it is possible to correct a number of deletions which is linear in the code length, with redundancy that is also linear in the code length. This is asymptotically optimal based on the bounds that were mentioned above.

What about correcting a small number, say a constant,

of deletions, a regime that is of interest in DNA storage since the number of deletions is small and the code length is moderately sized? As discussed previously, the optimal redundancy should be asymptotically between $t \log_2 n$ and $2t \log_2 n$ where t is the number of deletions. In the following, we discuss codes with close to optimal redundancy, using a fundamentally different idea from the concatenated code construction. Specifically, we discuss a generalization of the VT codes using an algebraic approach. Recall that the higher order weighted sum $\sum_{i=1}^n i^p c_i$, a natural generalization of the VT codes, is not guaranteed to provide a code correcting even two deletions. However, a similar higher order weighted sum is capable of correcting deletions for constrained sequences.

We illustrate the idea for the case of two deletions. In this case, an interesting observation is that if the codewords are sequences with at least a single 0 between any two 1's, then any codeword $(c_1, \dots, c_n)$ can be protected from two deletions by providing the sums $\sum_{i=1}^n (\sum_{j=1}^i j^p) c_i \mod 2n^{p+1}$ for $p = 0, 1, 2$.

Example 3. Consider the sequence 101001 of length 6, where any two 1's are separated by at least one 0. Its weighted modulo sums are

$$\begin{aligned} \sum_{i=1}^6 (\sum_{j=1}^i j^0) c_i \mod 2 \cdot 6^{0+1} &= 3 \\ \sum_{i=1}^6 (\sum_{j=1}^i j^1) c_i \mod 2 \cdot 6^{1+1} &= 28 \\ \sum_{i=1}^6 (\sum_{j=1}^i j^2) c_i \mod 2 \cdot 6^{2+1} &= 106. \end{aligned}$$

Suppose two deletions occur in 101001, resulting in 1001. We list all possible sequences that satisfy: (1) can result in 1001 after two deletions; and (2) there is at least one 0 bit between any two 1 bits. These sequences are given by 001001, 010001, 010010, 100001, 100010, 100100, 10100, 1, 010101, and 100101. It can be verified that only 101001 has the weighted modulo sums 3, 28, 106 as above.

But how do we guarantee at least one 0 between any two 1's? To this end, let us define an *indicator vector* $\mathbb{1}_{10}(c_1, \dots, c_n)$ of length n for a sequence $(c_1, \dots, c_n)$ as follows. The i 'th bit of $\mathbb{1}_{10}(c_1, \dots, c_n)$ is

$$\mathbb{1}_{10}(c_1, \dots, c_n)_i = \begin{cases} 1 & \text{if } c_i = 1, \text{ and } c_{i+1} = 0 \\ 0 & \text{otherwise.} \end{cases},$$

for $i \in \{1, \dots, n\}$, where it is assumed that $c_{n+1} = 1$. Note that by definition, for any binary sequence $(c_1, \dots, c_n)$, there is at least one 0 between any two 1's in $\mathbb{1}_{10}(c_1, \dots, c_n)$: it is impossible to have to consecutive 1's, since it would imply both that $c_i = 1$

and $c_{i+1} = 0$, and that $c_{i+1} = 1$ and $c_{i+2} = 0$, and clearly c_{i+1} cannot simultaneously be a 0 and a 1.

Therefore, for any sequence $(c_1, \dots, c_n)$, the indicator vector $\mathbb{1}_{10}(c_1, \dots, c_n)$ can be protected from two deletions given the weighted modulo sums $\sum_{i=1}^n (\sum_{j=1}^i j^p) \mathbb{1}_{10}(c_1, \dots, c_n)_i \bmod 2n^{p+1}$ for $p = 0, 1, 2$. Then, it suffices to protect the modulo sums $\sum_{i=1}^n (\sum_{j=1}^i j^p) \mathbb{1}_{10}(c_1, \dots, c_n)_i \bmod 2n^{p+1}$, $p = 0, 1, 2$ by using a very short repetition code, in order to recover the vector $\mathbb{1}_{10}(c_1, \dots, c_n)$. After recovering the vector $\mathbb{1}_{10}(c_1, \dots, c_n)$, one can protect a similarly defined indicator vector $\mathbb{1}_{01}(c_1, \dots, c_n)$. Finally, the sequence $(c_1, \dots, c_n)$ can be recovered from indicator vectors $\mathbb{1}_{10}(c_1, \dots, c_n)$ and $\mathbb{1}_{01}(c_1, \dots, c_n)$.

This approach extends to any constant number t of deletions by generalizing the observation: For any t , if the codewords are sequences that have at least $t - 1$ 0's between any two 1's, then the codewords can be protected from t deletions by using the weighted modulo sum $\sum_{i=1}^n (\sum_{j=1}^i j^p) c_i \bmod 3tn^{p+1}$ for $p = 0, 1, \dots, 6t$. Similarly, one can define indicator vectors such that the indicator vector of a sequence has at least $t - 1$ 0's between any two 1's. The resulting redundancy is then $4t \log_2 n + o(\log_2 n)$, which is asymptotically at most 4 times the optimal.

Despite the progress on codes correcting t deletions, several problems remain open. How to construct minimal-redundancy deletion codes which can also be decoded efficiently? How to approach or improve the existential redundancy bound? How to efficiently correct a combination of deletions and substitutions? The third problem is crucial in DNA storage applications as errors are normally a combination of deletions, insertions, and substitutions. Though the problem was investigated and codes combining the deletion codes above and the substitution codes were proposed, are there more redundancy efficient methods?

III. SLICED CHANNEL

One feature that fundamentally distinguishes DNA storage from traditional storage is that in traditional systems, the codeword is a single long sequence, whereas in DNA storage, the codeword is a set of unordered short sequences.

To see this, we briefly describe the workflow of DNA storage systems, shown in Fig 1. As mentioned earlier, in DNA storage information is encoded into sequences of four letters, A, C, G, T, which are synthesized into the respective DNA molecules. The synthesized DNA molecules are then placed in a solution inside a vial. In the reading phase, the sequences are amplified by a process called Polymerase Chain Reaction (PCR),

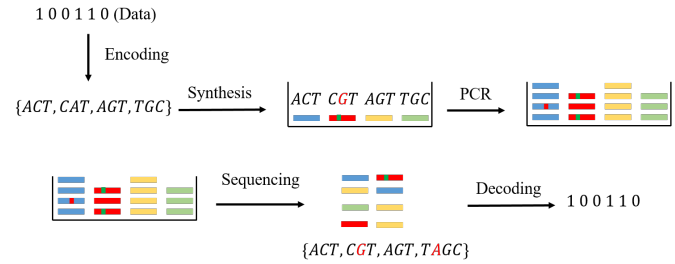


Fig. 1. An illustration of the processes in a typical in-vitro DNA storage system. The data (1, 0, 0, 1, 1, 0) is encoded into a set of short sequences {ACT, CAT, AGT, TGC} of nucleotides. Errors occur during the synthesis of the short sequences, turning the nucleotide A in CAT to G. The synthesized sequences, including the erroneous sequence "CGT", are amplified in the PCR process, generating multiple noisy copies of the synthesized sequences. During the sequencing process, these noisy copies are read and clustered such that each cluster consists of noisy copies of a synthesized sequence. Then, an estimate {ACT, CGT, AGT, TAGC} of the synthesized sequences is obtained from the clusters of noisy copies, where TAGC is an erroneous estimate of TGC. Finally, the estimated set of synthesized sequences is decoded into data.

which generates many more copies of the nucleotide sequences in the vial. The copies are then sampled and read through a sequencing process, producing many potentially erroneous copies of the sequences that were originally synthesized. By using clustering and reconstruction algorithms, the copies generated from the same sequence are clustered, and the corresponding sequence is reconstructed. Finally, the reconstructed sequences are decoded to retrieve the data. Due to technological constraints in the above processes, only short DNA molecules (≈ 100 nucleotides) can be synthesized and sequenced, meaning that information can only be encoded into a collection of short sequences. Moreover, the DNA molecules stored in the same vial are unordered; they all float in the same solution without any knowledge regarding which comes prior.

Note that errors occur in the collection of unordered short nucleotide sequences. This gives rise to the question of how to correct errors when the codeword is "sliced" into multiple unordered pieces. This brings new aspects to classic error correction setups, where the information is encoded into a codeword that is a single sequence, and retrieved from a noisy copy of that sequence. In the context of coding for DNA storage, the codeword is sliced into multiple unordered pieces, normally of equal lengths, which are presented both noisy and unordered to the decoder.

In the sliced codeword setting, we may either think of a codeword as a single sequence that it then sliced into multiple unordered pieces, or a priori consider the codeword as the set of those pieces; both approaches are

equivalent. In this paper, we choose the latter, i.e., we assume that the codeword is a set of M short sequences, each of length L . In existing DNA storage systems, L is of the order of magnitude 100 and M is of the order of magnitude 10^4 to 10^9 , based on the size of the data.

In DNA storage, the types of errors include: (1) deletions, insertions, and substitutions, that occur in either of the sequences, and were discussed in Section II. (2) sequence loss due to the fact that some DNA molecules may not be sampled during the sequencing process. As a result, the sequence contained in the DNA molecule is missing. The following is an example of coding over a set of short binary sequences.

Example 4. Assume that the given data is encoded to $M = 4$ sequences of length $L = 6$, say $\{110001, 100100, 101010, 111111\}$, which are placed together in a vial. Then, noisy copies of the codeword can be $\{100000, 1100001, 10101\}$, after deletion, insertion, substitution errors, and sequence loss that occur in any sequence in the set. Note that the identity of a noisy copy is not known. For example, the noisy copies 100000, 1100001, 10101 can be obtained from 110001, 100100, 101010, respectively, or from 100100, 110001, and 111111, respectively.

The setting of coding over a set of sequences can be considered as a generalization of the classic setting of coding over a single sequence, where the set contains only a single sequence and there is no sequence loss. Also, a similar ordering issue often arises in network packet transmission, where due to varied network delays and changes in routing, packets might arrive not in the same order they were sent.

To understand the problem of how to handle unordered sets of sequences, we focus on the basic setting where the codeword is a set of M different binary sequences of length L , and focus on substitution errors; more complex settings follow similar ideas. For example, one can transform a code that uses $\{0,1\}$ to one which uses A, C, G, T using the mapping mentioned in the introduction. Further, to correct deletion and insertion errors, one can combine the codes for this basic setting and the deletion codes discussed in Section II. To combat sequence loss, it is possible to add an “outer” code, such as the Reed-Solomon code.

Intuitively, correcting errors in the sliced codeword setting (over a set of sequences) is more difficult than correcting errors in a single sequence, since in the former setting, the information about the index of each sliced piece is lost. One natural way to correct errors in the set of sequences is to use error correction codes to protect each sequence independently. This is efficient when each

sequence roughly has the same amount of errors. In the case when some sequences have no errors, or some sequences have much more errors than average, the method may be inefficient since one has to protect every sequence from the largest number of errors possible.

To deal with the loss of the index information of each unordered sequence, another natural (and possibly the most common) approach is to use extra redundancy to index each sequence. That is, in each sequence, dedicate the first $\log_2 M$ (out of L) bits to record the index among the total M sequences. This gives an order to the sequences based on their indices, and reduces the problem of coding over an unordered set of sequences to that of coding over a single sequence.

It can be shown that the simple index-based scheme asymptotically approaches the best information rate, i.e., the channel capacity, for coding over an unordered set of sequences. More specifically, index-based schemes achieve the asymptotically optimal information rate in probabilistic settings where a fraction of sequence loss and substitution errors randomly occur. Partly for this reason, indexing schemes are used in most of the recent DNA storage experiments, where extra bases are dedicated to index each sequence, and Reed-Solomon codes are used to correct errors in the bases. In addition, many code constructions were proposed based on the indexing schemes.

One of the problems which need to be addressed for indexing schemes is to protect against errors in the index. One way is to encode the indices such that they are far from each other in Hamming distance (i.e., have many distinct bits). In this way, the indices are more robust to substitution errors. However, it requires more redundancy in the indices. To resolve this issue, another approach is to use data to protect errors in indices. When the Hamming distance between two indices is small, meaning that they are ambiguous under substitution errors, it is required the data in the corresponding two sequences have a large enough Hamming distance. With this constraint, the decoder can distinguish two sequences based on their data, if it fails to decode their indices.

While index-based schemes achieve asymptotically the optimal information rate in probabilistic settings, how do they perform in deterministic settings? In deterministic settings, the number of errors is bounded and zero-error decoding is required. When the number of errors is not large, it is reasonable to look at the redundancy, rather than the information rate of a scheme, which approaches one with a small number of errors. Note that different from information rate, which measures the ratio between the amount of information and the number of

symbols used to store the information, redundancy measures the difference between the two. But how should we define “redundancy” in the unordered sequence setting? To study this question, we define the redundancy of a code as $\log_2 \binom{2^L}{M} - \log_2 |\mathcal{C}|$, where $|\mathcal{C}|$ is the size of the code (i.e., number of codewords) and $\log_2 |\mathcal{C}|$ is the number of information bits the code can represent. This definition measures how many extra bits are needed for error correction, and would be zero in the case of no errors. Under this definition, the extra redundancy needed for indexing in an index-based scheme is at least $\log_2 \binom{2^L}{M} - M(L - \log_2 M)$ (which is linear in M) even when there are no errors.

Can one use less redundancy than that? Using counting arguments, one can show that the optimal redundancy for correcting a total number of t substitution errors across all M unordered sequences of length L is at most $2t \log_2(ML) + o(\log_2(ML))$ for small t (e.g., a constant), and at most linear in $t \log_2(\frac{ML}{t})$ for large t (e.g., a fraction of ML). In addition, the optimal redundancy is at least $t \log_2(ML) + o(\log_2(ML))$ for small t , and at least linear in $t \log_2(\frac{ML}{t})$ for large t . The upper and lower bounds are order-wise the same. Note that in the classic setting of correcting t substitution errors over a single sequence of length ML (which is equivalent to M ordered sequences of length L), similar counting arguments also yield $2t \log_2(ML) + o(\log_2(ML))$ for small t , and linear in $t \log_2(\frac{ML}{t})$ for large t . In addition, the lower bound on redundancy is order-wise the same as the upper bound. This result has a surprising implication: **it costs almost the same amount of redundancy to correct errors over a set of unordered and ordered sequences!** This is highly surprising, since the unordered case is intuitively more complex.

We now compare the redundancy bound $2t \log_2(ML) + o(\log_2(ML))$ to that of the index-based schemes presented earlier, which is linear in M . Since M is much larger than L , the redundancy in the index-based schemes is much larger than bound $2t \log_2(ML) + o(\log_2(ML))$ whenever $t = o(\frac{M}{\log_2(ML)})$. In what follows we describe ideas that close this gap, i.e., provide codes correcting substitution errors with almost optimal redundancy, and thus are better than index-based schemes for this many substitution errors.

The idea is to use the data itself for the purpose of indexing, or alternatively, encoding data inside the index. Specifically, we use the lexicographic order of the data for indexing, that is, the prefix in each sequence is used for indexing, while also containing data.

Example 5. Let the codeword be $\{1001101, 0101100, 1010001, 0001001\}$, where the

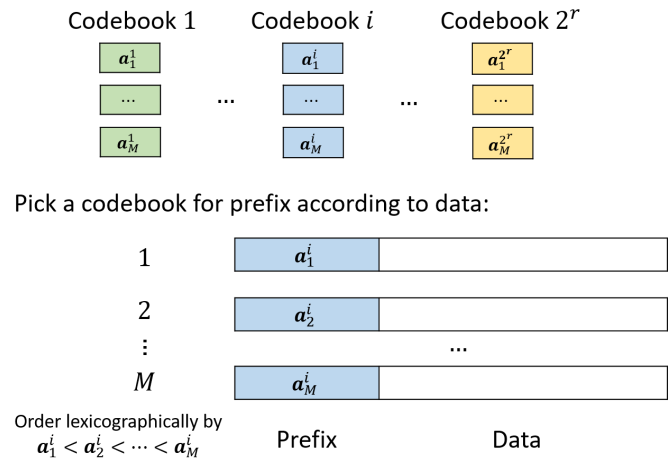


Fig. 2. An illustration of codes that use data for indexing.

first three bits are the prefix in each sequence used for indexing. Then one can order the set of sequences in the codeword by 0001001, 0101100, 1001101, 1010001 in ascending lexicographic order of the prefixes.

Using prefixes for indexing is similar to index-based schemes. Yet, unlike index-based schemes, the prefixes also encode information. In order to make the indexing of the prefixes robust from substitution errors, the collection of prefixes in all sequences constitutes a code with large minimum Hamming distance. This is similar to protecting the indices from errors in the index-based schemes. The difference is that the indexing prefixes here encode information. Information is encoded into prefixes through different choices of the codes, as shown in Fig. 2. The construction of codes for the prefixes can be done using a greedy algorithm, so that the prefixes in the code are generated bit-by-bit.

The scheme of using data to index avoids the index bits, and achieves redundancy that is linear in $t \log_2(ML)$, i.e., almost optimal. One can also combine it with the deletion correcting codes discussed in Section II, and enable deletion and/or insertion correction as well.

Finally, some questions about the unordered setting remain unanswered: What is the optimal redundancy for a large number of errors? How to construct efficient codes that achieve order-wise optimal redundancy?

IV. DUPLICATION

Unlike previous sections, this section is motivated by storing information in the DNA of living organisms. The process involves synthesizing DNA sequences which are then inserted into the DNA of living organisms. We can then sequence the DNA extracted from these organisms,

or more likely, their descendants, to read the information. Thus, the DNA storage channel in this case corrupts data not only due to synthesis and sequencing errors, but also due to naturally occurring biological processes that mutate the DNA.

We now focus on the errors (mutations) introduced by biological processes. It is well known that when cells divide, the genetic material is replicated. However, the DNA replication process is not without noise, and the resulting copy may be corrupted by several error types. These include substitution, where a base is replaced by another (point mutation), as well as insertions and deletions of blocks. These types of errors have been studied to various extents by existing literature. Another type of error is duplication, whereby a copy of a substring of the DNA is inserted. Duplications accumulate over time, and it has been found that the majority of human DNA is duplicated. Since this error type is rarely found in electronic communication, it has not been studied in the coding theory community, and in what follows, we focus on it solely.

What kind of duplications are possible? Several biological mechanisms are known to create duplications in the process of replicating the DNA. We illustrate a few here. Perhaps the simplest one (though not necessarily the most common) is *tandem duplication*. This mutation process takes a substring and inserts a duplicate of it immediately after its original location, for example,

$$\text{ACCTAGGA} \Rightarrow \text{ACCTA}\overline{\text{CTAGGA}} \quad (\text{tandem}),$$

where the underlined part is the substring being duplicated, and the overlined part is the inserted duplication. In *interspersed duplication*, the duplicated part is inserted anywhere in the sequence, for example

$$\text{ACCTAGGA} \Rightarrow \text{ACCTAGG}\overline{\text{CTAA}} \quad (\text{interspersed}).$$

Another duplication process, called *reverse-complement duplication* (r.c. duplication), takes the substring to duplicate, and inserts a reversed and complemented duplicate of it immediately after its location, for example

$$\text{ACCTAGGA} \Rightarrow \text{ACCTA}\overline{\text{TAGGGA}} \quad (\text{r.c.}).$$

Here we use the Watson-Crick base pairing, making A and T complements of each other, and similarly, C and G. All of the processes mentioned above are the result of known biological mutation processes whose mechanisms we understand. For the sake of mathematical simplicity, and to better illustrate the intricacies of duplication processes, we introduce an artificial duplication process called *end duplication* which inserts the duplicated part at the end of the sequence, for example,

$$\text{ACCTAGGA} \Rightarrow \text{ACCTAGG}\overline{\text{ACTA}} \quad (\text{end}).$$

As a final note on duplication processes, we emphasize that following a duplication process, another may occur, perhaps of a different type, and perhaps of a different length. Thus, over time, duplications accumulate, like layers of an onion. A naive inspection of a DNA sequence may only reveal the outer layer, namely, the last duplications made, and only after removing those, older occurrences become visible.

We can now formalize the description of the duplication channel. We shall be working over some finite alphabet Σ (which in the case of DNA molecules will be $\Sigma = \{A, C, G, T\}$). We store a sequence $x \in \Sigma^*$ where Σ^* is the set of all finite length sequences over Σ . The channel then applies any number of duplications, resulting in a string $y \in \Sigma^*$. We denote this process as $x \Rightarrow^* y$. The set of all possible mutated outcomes, given that x was stored, is called the *descendant cone* of x , and is denoted by $D^*(x)$. Conversely, the set of all possible strings that may be mutated by the channel into y is called the *ancestor cone* of y , and is denoted by $A^*(y)$.

When faced with such a channel, our goal is to construct error-correcting codes that can undo the duplications and recover the original stored sequence. General coding-theoretic principles guide us to define an error-correcting code as a set of sequences $C \subseteq \Sigma^n$, whose descendant cones are disjoint, namely, for any $c, c' \in C$, $D^*(c) \cap D^*(c') = \emptyset$. Thus, any corrupted sequence belongs to a single descendant cone of a valid codeword, and the decoding process simply outputs that codeword in response.

Many questions arise: How do we find a good error-correcting code? What makes a good error-correcting code? What is the best possible? How do we encode, and how do we decode? How does the answer depend on the type and parameters of the duplication processes? In what follows we briefly outline partial answers to these questions, and along the way, uncover connections to other motivating problems.

A. Know thy enemy – Understanding descendant cones

The first property of interest, when studying descendant cones, is knowing their size. Since our ultimate goal is constructing error-correcting codes, which are equivalent to packing descendant cones without overlap, finding their size may help us bound the parameters of such codes. The number of strings in any descendant cone is obviously infinite, and thus, we do not measure their size but rather the rate at which they grow with each mutation step. This property is called the *capacity*, and sometimes the *combinatorial entropy*.

Formally speaking, to compute the capacity of the descendant cone of $x \in \Sigma^*$, the definition calls for

counting the number of descendants of length n , i.e., $|D^*(x) \cap \Sigma^n|$. Taking $\log_2$ of this number and dividing by n gives us the exponential growth rate we are after. Thus,

$$\text{cap}(x) = \limsup_{n \rightarrow \infty} \frac{1}{n} \log_2 |D^*(x) \cap \Sigma^n|.$$

A large capacity indicates a fast growing descendant cone, and similarly, a small capacity indicates slow growth. Packing fast growing descendant cones may be more difficult, resulting in smaller, less efficient, error-correcting codes.

The capacity may obviously depend on the alphabet size, the starting sequence x , and the duplication rules. To illustrate the subtleties of the latter, fix an alphabet Σ and a starting sequence x . First consider the end-duplication system, in which each mutation copies a fixed-length substring of length k to the end. It has been shown that this has full capacity, i.e., $\text{cap}_k^{\text{end}}(x) = \log_2 |\Sigma|$, which is the highest possible value the capacity may have, indicating the highest possible growth rate for a descendant cone. We now tweak a single parameter – instead of end duplication, we consider tandem duplication, namely the duplicated sequence of fixed length k is inserted immediately after its original position. With this minute change, the capacity vanishes completely, i.e., $\text{cap}_k^{\text{tan}}(x) = 0$, indicating sub-exponential growth of the descendant cone.

One might argue that the capacity is perhaps too harsh: it takes into account what is *possible*, where instead it should take into account what is *probable*. We can describe the mutations as a stochastic process. We start with the initial sequence $x \in \Sigma^*$, and then at each round, a randomly chosen duplication rule (duplicating substring of fixed length k) is applied to a randomly selected position. We can set, to our liking, the distributions from which the duplication rule and the location are chosen. We denote the resulting sequence after n mutations by $S_n(x)$, and observe that it is a random variable. We can then define the *entropy* of $S_n(x)$ as

$$H(S_n(x)) = - \sum_{w \in \Sigma^*} \Pr(S_n(x) = w) \log_2 \Pr(S_n(x) = w).$$

With this, the *entropy rate* of the entire system

$$h(S(x)) = \limsup_{n \rightarrow \infty} \frac{1}{n} H(S_n(x)).$$

Loosely speaking, $h(S(x))$ measures the amount of information generated by an application of a random duplication rule. Using standard information-theoretic

arguments one can show that the capacity bounds the entropy rate from above, namely,

$$h(S(x)) \leq \text{cap}(x).$$

Once again we demonstrate the intricacies of string-duplication systems by showing how even the smallest of changes create dramatically different results. For the sake of this demonstration we focus on the reverse-complement string duplication system over the binary alphabet $\Sigma = \{0, 1\}$. We further assume for simplicity that the initial string is $x = 0$, all duplications are of the same fixed length $k = 1$, and that their location is chosen uniformly and independently in each round. We emphasize that the fact the locations are chosen uniformly does not mean that $S_n(x)$ is distributed uniformly. For example, there is only one way of deriving 0111 from $x = 0$,

$$0 \Rightarrow \underline{0}\bar{1} \Rightarrow \underline{0}\bar{1}1 \Rightarrow \underline{0}\bar{1}11,$$

and the probability of this happening is exactly $1 \cdot \frac{1}{2} \cdot \frac{1}{3} = \frac{1}{6}$. However, there are two ways of deriving 0101

$$0 \Rightarrow \underline{0}\bar{1} \Rightarrow \underline{0}\bar{1}1 \Rightarrow 0\underline{1}\bar{0}1,$$

$$0 \Rightarrow \underline{0}\bar{1} \Rightarrow 0\underline{1}\bar{0} \Rightarrow 01\underline{0}\bar{1},$$

and we get 0101 with probability $\frac{1}{3}$. Interestingly, the entropy-rate we are after is connected to the asymptotics of permutation signatures, and the best we know is that for duplication length $k = 1$,

$$0.8689 \leq h(S(x = 0)) \leq 0.9067 < \text{cap}(x = 0) = 1.$$

Importantly, while the capacity is full (i.e., “most” sequences are obtainable via carefully chosen derivation paths), not all outcomes are probable, and the entropy rate is strictly less than 1. To further complicate matters, consider duplication of length $k = 2$ and an equally long starting sequence, $x = 00$. In this case, it has been shown that

$$0 = h(S(x = 00)) < \text{cap}(x = 00) = 1.$$

Namely, while again, “most” sequences are obtainable, only very few are probable. This surprising result was obtained by proving that with high probability, $S_n(x = 00)$ is eventually almost entirely an alternating sequence of 0101 A simulation of this fact is shown Figure 3.

The last two properties we shall describe are perhaps more interesting from a bio-informatics perspective. Consider the following question: We are given some distant ancestor that humans evolved from, and this ancestor does not have the DNA substring that codes for a specific protein humans have. Can tandem duplication *alone* mutate the ancestor’s DNA sequence into a sequence that contains the instructions for that protein? In our

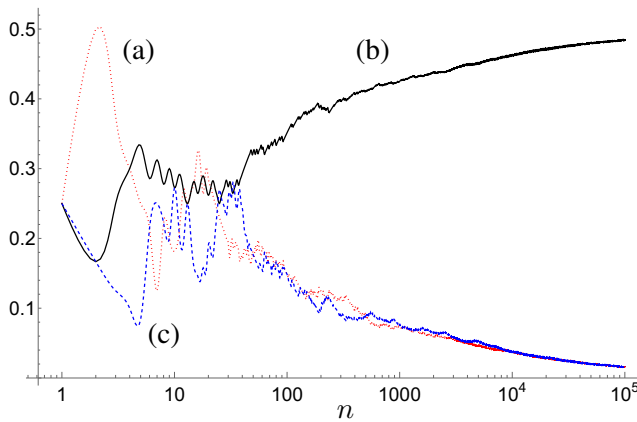


Fig. 3. An example simulation of the reverse-complement string-duplication system with starting sequence $x = 00$, and duplication length $k = 2$, showing for $S_n(00)$ the frequencies of the substrings (a) 00, (b) 01 (which equals that of 10), and (c) 11.

mathematical framework of string-duplication systems, we say a system with a starting sequence $x \in \Sigma^*$ is *fully expressive* if any given sequence $y \in \Sigma^*$ appears as a substring of some descendant of x . Returning to our previous example of end duplication vs. tandem duplication (both of some fixed length k), it was shown that tandem duplication is not fully expressive, whereas end duplication is. To put this in context, imagine the following challenge: We are given Tolstoy's "War and Peace" (which we consider as a very long sequence of symbols). We can duplicate substrings of some fixed length, say, $k = 200$. Our goal is to create a substring which is Shakespeare's "Macbeth". When the duplicated parts are inserted next to their original position (tandem duplication), this is impossible. However, when the duplicated parts are placed at the end, the challenge is solvable! (albeit, the procedure might be extremely lengthy)

The final property we would like to mention is that of distance to the root. When reading a mutated version $y \in \Sigma^*$ of the stored sequence $x \in \Sigma^*$, our goal is to reverse the mutation process and find x . This may be performed by undoing duplications, a process called *de-duplication*. The process stops when no further de-duplications are possible, and the sequence at that point is called a *root* of y . The number of de-duplication steps is called the distance to a root from y . In the binary case $\Sigma = \{0, 1\}$, with unbounded tandem duplication, the root is always one of six options $\{0, 1, 01, 10, 010, 101\}$. In this setting, we let $f(n)$ denote the maximum distance to the root of a binary sequence of length n . Surprisingly,

$$0.045 \leq \lim_{n \rightarrow \infty} \frac{f(n)}{n} \leq 0.4,$$

and the lower bound in fact holds for all but an exponentially small fraction of sequences of length n . Thus, the vast majority of sequences have a distance to the root that is linear in their length. This result remains essentially the same even if the duplication process is imprecise.

B. Constructing error-correcting codes

Armed with a better understanding of descendant cones, we may now approach the problem of designing error-correcting codes for string-duplication channels. Two main issues are of interest: finding a good code (i.e., making sure descendant cones of distinct codewords are disjoint), and finding an efficient decoding algorithm. Throughout this section, we shall consider the tandem-duplication string-duplication channel as an example.

When the duplication length is fixed at some length k , we are indeed fortunate. If the alphabet contains exactly q letters, we may assume without loss of generality that $\Sigma = \mathbb{Z}_q$, namely, the ring of integers with addition modulo q . It has been suggested to view any string after taking the k -step discrete derivative ∂_k , i.e., for each i , subtracting the letter in position $i - k$ from the letter in position i . Thus, $\partial_k x = x0^k - 0^k x$, where 0^k denotes a run of k zeros, and subtraction is symbol-wise over $\mathbb{Z}_q$. This operation is invertible. Moreover, in the derivative domain, a tandem duplication of length k manifests as an insertion of 0^k . As a consequence, we obtain the following:

- Any sequence $x \in \Sigma^*$ has a unique root.
- The unique root of a sequence x may be reached by de-duplications performed in any order.
- Two sequences, x and x' , have intersecting descendant cones if and only if they have the same root.

We note in passing that these assertions are not true even if we relax our setting minutely. For example, if we allow de-duplications of any length (instead of a fixed length) then the sequence 210121010 does not have a unique root:

$$\begin{aligned} 210121010 &\longrightarrow 21010 \longrightarrow 210, \\ 210121010 &\longrightarrow 2101210, \end{aligned}$$

where $\longrightarrow$ denotes a de-duplication, and the underlined part is de-deuplicated.

Returning to our search for codes that correct tandem duplications of fixed length k , the three properties listed above lead us to the following solution: Construct a code by taking as codewords all the irreducible sequences of length n over $\mathbb{Z}_q$ (where an irreducible sequence is a sequence that is its own root, namely, it does not contain any duplications of length k). With a small tweak

this code can be made optimal, and allows information storage at a rate of

$$\log_2 q - \frac{(q-1)\log_2 e}{q^{k+1}}(1 + o(1)).$$

Decoding is simple, since by the properties above, we can de-duplicate in any order we wish, until reaching the unique root which must be the transmitted sequence.

In essence, since the duplications introduced by biological processes are part of the evolutionary process, we can think of the error-correcting codes we described as evolution-correcting codes.

V. BIBLIOGRAPHIC NOTES

There are several implementations [3], [8], [9], [13], [20], [23], [30], [36], [37], [76], [85], [98], [116], [117], [122] of in-vitro and in-vivo DNA storage demonstrating their potential and motivating the error and channel models considered in this paper. A detailed description of the errors and the channel can be found in [44], [59]. We also refer to [27], [87], [118] for a broader overview of different aspects in DNA storage.

A. Deletion Codes

The study of codes correcting deletions and insertions was introduced in the seminal papers [68], [83], where it was shown in [68] that deletion codes correct a combination of deletions and insertions. The upper and lower bounds on the optimal redundancy of deletion codes were also given in [68]. The VT codes were proposed in [108]. An algebraic generalization of VT codes with redundancy linear in code length was presented in [46] and further extended in [43]. The first codes correcting a number of linear in code length deletions based on concatenated code structures were proposed in [82] and were improved in [39], [40]. Using the concatenated code construction, [10] proposed the first codes correcting a small number of deletions with redundancy logarithmic in the code length. For two deletions, the result in [10] was improved by [35], [38], [91]. The first order-wise optimal codes correcting a constant number of deletions were given by [17], [89], [90].

The algebraic generalizations of VT codes discussed in Section II for correcting deletions were presented in [89]–[91]. Compared to the VT generalizations in [43], [46] that require linear redundancy, the generalizations in [89]–[91] are capable of correcting a constant number of deletions with asymptotically at most 4 times the optimal redundancy, which is a logarithm of the code length. Combining the codes in [90] and the VT codes, [96] further improved the redundancy in [90] from

$4t \log n + o(\log n)$ to $(4t - 1) \log n + o(\log n)$, where t is the number of deletions and n is the code length. Besides the above code constructions, existential bounds improving the results in [68] were recently presented in [2].

Other related problems include: systematic deletion codes [5], [17], [21], [41], [77], [90], non-binary deletion codes [22], [42], [61], [69], [70], [89], [106], deletion codes with randomized decoding [6], [12], [47], [50], channel capacity of deletion channels [18], [24], [25], [28], [51], [52], [54], [99], [107], [109], codes correcting a combination of deletions, insertions, substitutions, and transpositions [11], [16], [33], [35], codes correcting a burst of deletions [62], [81], [97], [111], codes for sticky insertions [26], [72], and codes correcting asymmetric deletions [100], [112]. In addition, the application of edit distance in natural language processing and biological data analysis can be found in [80], [115]. See [19], [74], [75], [94] for a broader review of this topic.

B. Sliced Channel

The model of encoding information into a set of unordered and equal-length sequences was introduced in [45], where it was shown that index-based schemes achieve the channel capacity when there are sequence losses. Later, the channel capacity analysis was extended to channels with both sequence loss and substitution errors [65], [66], [86], [114]. The protection of indices against errors assuming index-based schemes was addressed in [63], [84], [95].

The definition of redundancy measuring the extra redundancy needed for error protection was introduced in [64], where it was shown that the redundancy for index-based schemes is linear in the number of sequences. The order-wise optimal redundancy under this definition was obtained in [93], where the idea of using data for indexing was proposed. By using this idea, code constructions were presented in [92] to achieve order-wise optimal redundancy and in [113] to obtain improved results upon those in [64].

Other related problems include: permutation channels [55], [56], [60], [73], [101], [110], codes for reconstruction from substrings [14], [34], [53], [79], [119], torn-paper channels [4], [78], [88].

C. Duplication

Early attempts of in-vivo information storage can be found in [23], [116]. Proofs of concepts for storing information in the DNA of living organisms were provided in [85] and [122]. In [59], it was pointed out that the majority of the human DNA contains duplications. The

string-duplication channels were introduced in [31] and the capacity and expressiveness of several duplication rules were studied. These results were extended in [48], [57]. The stochastic channel model, called a Pólya string model, was introduced in [29], and further studied in [7], [32], [71]. In particular, [32] developed a parameter-estimation scheme based on this model. The distance to the root in tandem duplication channels was studied in [1].

Error-correcting codes for string-duplication channels were first studied in [49]. The work was followed by many others, among them works studying: tandem duplication [15], [58], [67], [123], [124], Levenshtein reconstruction for uniform tandem duplication [120], [121], noisy tandem duplication [102]–[105], palindromic duplications [67], [124], and reverse-complement duplications [7].

VI. ACKNOWLEDGMENTS

This research was partially supported by NSF grant CCF-1816965 and NSF grant CCF-1717884.

REFERENCES

- [1] N. Alon, J. Bruck, F. Farnoud, and S. Jain, "Duplication distance to the root for binary sequences," *IEEE Transactions on Information Theory*, vol. 63, no. 12, pp. 7793–7803, Dec. 2017.
- [2] N. Alon, G. Bourla, B. Graham, X. He, and N. Kravitz, "Logarithmically larger deletion codes of all distances," *IEEE Transactions on Information Theory*, 2023.
- [3] P. L. Antkowiak, J. Lietard, M. Z. Darestani, M. M. Somoza, W. J. Stark, R. Heckel, and R. N. Grass, "Low cost DNA data storage using photolithographic synthesis and advanced information reconstruction and error correction," *Nature Communications*, vol. 11, no. 1, p. 5345, 2020.
- [4] D. Bar-Lev, S. M. E. Yaakobi, and Y. Yehezkeally, "Adversarial torn-paper codes," *IEEE Transactions on Information Theory*, 2023.
- [5] D. Belazzougui, "Efficient deterministic single round document exchange for edit distance," *arXiv preprint arXiv:1511.09229*, 2015.
- [6] D. Belazzougui and Q. Zhang, "Edit distance: Sketching, streaming, and document exchange," in *2016 IEEE 57th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 2016, pp. 51–60.
- [7] E. Ben-Tolila and M. Schwartz, "On the reverse-complement string-duplication system," *IEEE Transactions on Information Theory*, vol. 68, no. 11, pp. 7184–7197, Nov. 2022.
- [8] M. Blawat, K. Gaedke, I. Huetter, X.-M. Chen, B. Turczyk, S. Inverso, B. W. Pruitt, and G. M. Church, "Forward error correction for DNA data storage," *Procedia Computer Science*, vol. 80, pp. 1011–1022, 2016.
- [9] J. Bornholt, R. Lopez, D. M. Carmean, L. Ceze, G. Seelig, and K. Strauss, "A DNA-based archival storage system," in *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems*, 2016, pp. 637–649.
- [10] J. Brakensiek, V. Guruswami, and S. Zbarsky, "Efficient low-redundancy codes for correcting multiple deletions," *IEEE Transactions on Information Theory*, vol. 64, no. 5, pp. 3403–3410, 2017.
- [11] K. Cai, Y. M. Chee, R. Gabrys, H. M. Kiah, and T. T. Nguyen, "Correcting a single indel/edit for DNA-based data storage: Linear-time encoders and order-optimality," *IEEE Transactions on Information Theory*, vol. 67, no. 6, pp. 3438–3451, 2021.
- [12] D. Chakraborty, E. Goldenberg, and M. Koucký, "Low distortion embedding from edit to hamming distance using coupling," in *Electronic Colloquium on Computational Complexity*, vol. 22, 2015, p. 111.
- [13] S. Chandak, K. Tatwawadi, B. Lau, J. Mardia, M. Kubit, J. Neu, P. Griffin, M. Wootters, T. Weissman, and H. Ji, "Improved read/write cost tradeoff in DNA-based data storage using ldpc codes," in *2019 57th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*. IEEE, 2019, pp. 147–156.
- [14] Z. Chang, J. Chrisnata, M. F. Ezerman, and H. M. Kiah, "Rates of DNA sequence profiles for practical values of read lengths," *IEEE Transactions on Information Theory*, vol. 63, no. 11, pp. 7166–7177, 2017.
- [15] Y. M. Chee, J. Chrisnata, H. M. Kiah, and T. T. Nguyen, "Efficient encoding/decoding of GC-balanced codes correcting tandem duplications," *IEEE Transactions on Information Theory*, vol. 66, no. 8, pp. 4892–4903, Aug. 2020.
- [16] K. Cheng, Z. Jin, X. Li, and K. Wu, "Block edit errors with transpositions: Deterministic document exchange protocols and almost optimal binary codes," *arXiv preprint arXiv:1809.00725*, 2018.
- [17] —, "Deterministic document exchange protocols and almost optimal binary codes for edit errors," *Journal of the ACM*, vol. 69, no. 6, pp. 1–39, 2022.
- [18] M. Cheraghchi, "Capacity upper bounds for deletion-type channels," *Journal of the ACM (JACM)*, vol. 66, no. 2, pp. 1–79, 2019.
- [19] M. Cheraghchi and J. Ribeiro, "An overview of capacity results for synchronization channels," *IEEE Transactions on Information Theory*, vol. 67, no. 6, pp. 3207–3232, 2020.
- [20] G. M. Church, Y. Gao, and S. Kosuri, "Next-generation digital information storage in DNA," *Science*, vol. 337, no. 6102, pp. 1628–1628, 2012.
- [21] G. Cormode, M. S. Paterson, S. C. Sahinalp, and U. Vishkin, "Communication complexity of document exchange," 1999.
- [22] D. Cullina and N. Kiyavash, "An improvement to levenshtein's upper bound on the cardinality of deletion correcting codes," *IEEE Transactions on Information Theory*, vol. 60, no. 7, pp. 3862–3870, 2014.
- [23] J. Davis, "Microvenus," *Art Journal*, vol. 55, no. 1, pp. 70–74, 1996.
- [24] S. N. Diggavi and M. Grossglauser, "On transmission over deletion channels," in *Proceedings of the Annual Allerton Conference on Communication Control and Computing*, vol. 39, no. 1. Citeseer, 2001, pp. 573–582.
- [25] R. L. Dobrushin, "Shannon's theorems for channels with synchronization errors," *Problemy Peredachi Informatsii*, vol. 3, no. 4, pp. 18–36, 1967.
- [26] L. Dolecek and V. Anantharam, "Repetition error correcting sets: Explicit constructions and prefixing methods," *SIAM Journal on Discrete Mathematics*, vol. 23, no. 4, pp. 2120–2146, 2010.
- [27] Y. Dong, F. Sun, Z. Ping, Q. Ouyang, and L. Qian, "DNA storage: research landscape and future prospects," *National Science Review*, vol. 7, no. 6, pp. 1092–1107, 2020.

- [28] E. Drinea and M. Mitzenmacher, "Improved lower bounds for the capacity of iid deletion and duplication channels," *IEEE Transactions on Information Theory*, vol. 53, no. 8, pp. 2693–2714, 2007.
- [29] O. Elishco, F. Farnoud, M. Schwartz, and J. Bruck, "The entropy rate of some Pólya string models," *IEEE Transactions on Information Theory*, vol. 65, no. 12, pp. 8180–8193, Dec. 2019.
- [30] Y. Erlich and D. Zielinski, "DNA fountain enables a robust and efficient storage architecture," *Science*, vol. 355, no. 6328, pp. 950–954, 2017.
- [31] F. Farnoud, M. Schwartz, and J. Bruck, "The capacity of string-duplication systems," *IEEE Transactions on Information Theory*, vol. 62, no. 2, pp. 811–824, Feb. 2016.
- [32] —, "Estimation of duplication history under a stochastic model for tandem repeats," *BMC Bioinformatics*, vol. 20, no. 64, pp. 1–11, Feb. 2019.
- [33] R. Gabrys, V. Guruswami, J. Ribeiro, and K. Wu, "Beyond single-deletion correcting codes: substitutions and transpositions," *IEEE Transactions on Information Theory*, vol. 69, no. 1, pp. 169–186, 2022.
- [34] R. Gabrys and O. Milenkovic, "Unique reconstruction of coded sequences from multiset substring spectra," in *2018 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2018, pp. 2540–2544.
- [35] R. Gabrys, E. Yaakobi, and O. Milenkovic, "Codes in the Damerau distance for deletion and adjacent transposition correction," *IEEE Transactions on Information Theory*, vol. 64, no. 4, pp. 2550–2570, 2017.
- [36] N. Goldman, P. Bertone, S. Chen, C. Dessimoz, E. M. LeProust, B. Sipos, and E. Birney, "Towards practical, high-capacity, low-maintenance information storage in synthesized DNA," *Nature*, vol. 494, no. 7435, pp. 77–80, 2013.
- [37] R. N. Grass, R. Heckel, M. Puddu, D. Paunescu, and W. J. Stark, "Robust chemical preservation of digital information on DNA in silica with error-correcting codes," *Angewandte Chemie International Edition*, vol. 54, no. 8, pp. 2552–2555, 2015.
- [38] V. Guruswami and J. Håstad, "Explicit two-deletion codes with redundancy matching the existential bound," *IEEE Transactions on Information Theory*, vol. 67, no. 10, pp. 6384–6394, 2021.
- [39] V. Guruswami and R. Li, "Efficiently decodable insertion/deletion codes for high-noise and high-rate regimes," in *2016 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2016, pp. 620–624.
- [40] V. Guruswami and C. Wang, "Deletion codes in the high-noise and high-rate regimes," *IEEE Transactions on Information Theory*, vol. 63, no. 4, pp. 1961–1970, 2017.
- [41] B. Haeupler, "Optimal document exchange and new codes for insertions and deletions," in *2019 IEEE 60th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 2019, pp. 334–347.
- [42] B. Haeupler and A. Shahrashbi, "Synchronization strings: codes for insertions and deletions approaching the singleton bound," in *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing*, 2017, pp. 33–46.
- [43] M. Hagiwara, "On ordered syndromes for multi insertion/deletion error-correcting codes," in *2016 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2016, pp. 625–629.
- [44] R. Heckel, G. Mikutis, and R. N. Grass, "A characterization of the DNA data storage channel," *Scientific Reports*, vol. 9, no. 1, pp. 1–12, 2019.
- [45] R. Heckel, I. Shomorony, K. Ramchandran, and N. David, "Fundamental limits of DNA storage systems," in *2017 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2017, pp. 3130–3134.
- [46] A. S. Helberg and H. C. Ferreira, "On multiple insertion/deletion correcting codes," *IEEE Transactions on Information Theory*, vol. 48, no. 1, pp. 305–308, 2002.
- [47] U. Irmak, S. Mihaylov, and T. Suel, "Improved single-round protocols for remote file synchronization," in *Proceedings IEEE 24th Annual Joint Conference of the IEEE Computer and Communications Societies.*, vol. 3. IEEE, 2005, pp. 1665–1676.
- [48] S. Jain, F. Farnoud, and J. Bruck, "Capacity and expressiveness of genomic tandem duplication," *IEEE Transactions on Information Theory*, vol. 63, no. 10, pp. 6129–6138, Oct. 2017.
- [49] S. Jain, F. Farnoud, M. Schwartz, and J. Bruck, "Duplication-correcting codes for data storage in the DNA of living organisms," *IEEE Transactions on Information Theory*, vol. 63, no. 8, pp. 4996–5010, Aug. 2017.
- [50] H. Jowhari, "Efficient communication protocols for deciding edit distance," in *Algorithms—ESA 2012: 20th Annual European Symposium, Ljubljana, Slovenia, September 10–12, 2012. Proceedings 20*. Springer, 2012, pp. 648–658.
- [51] A. Kalai, M. Mitzenmacher, and M. Sudan, "Tight asymptotic bounds for the deletion channel with small deletion probabilities," in *2010 IEEE International Symposium on Information Theory*. IEEE, 2010, pp. 997–1001.
- [52] Y. Kanoria and A. Montanari, "Optimal coding for the binary deletion channel with small deletion probability," *IEEE Transactions on Information Theory*, vol. 59, no. 10, pp. 6192–6219, 2013.
- [53] H. M. Kiah, G. J. Puleo, and O. Milenkovic, "Codes for DNA sequence profiles," *IEEE Transactions on Information Theory*, vol. 62, no. 6, pp. 3125–3146, 2016.
- [54] A. Kirsch and E. Drinea, "Directly lower bounding the information capacity for channels with iid deletions and duplications," *IEEE Transactions on Information Theory*, vol. 56, no. 1, pp. 86–102, 2009.
- [55] M. Kovačević and V. Y. Tan, "Codes in the space of multisets—coding for permutation channels with impairments," *IEEE Transactions on Information Theory*, vol. 64, no. 7, pp. 5156–5169, 2018.
- [56] M. Kovačević and D. Vukobratović, "Perfect codes in the discrete simplex," *Designs, Codes and Cryptography*, vol. 75, pp. 81–95, 2015.
- [57] M. Kovačević, "Zero-error capacity of duplication channels," *IEEE Transactions on Communications*, vol. 67, no. 10, pp. 6735–6742, 2019.
- [58] M. Kovačević and V. Y. F. Tan, "Asymptotically optimal codes correcting fixed-length duplication errors in DNA storage systems," *IEEE Communications Letters*, vol. 22, no. 11, pp. 2194–2197, 2018.
- [59] E. S. Lander, L. M. Linton, B. Birren, C. Nusbaum, M. C. Zody, J. Baldwin, K. Devon, K. Dewar, M. Doyle, W. FitzHugh *et al.*, "Initial sequencing and analysis of the human genome," *Nature*, vol. 409, no. 6822, pp. 860–921, 2001.
- [60] M. Langberg, M. Schwartz, and E. Yaakobi, "Coding for the ℓ_∞ -limited permutation channel," *IEEE Transactions on Information Theory*, vol. 63, no. 12, pp. 7676–7686, 2017.
- [61] T. A. Le and H. D. Nguyen, "New multiple insertion-deletion correcting codes for non-binary alphabets," *arXiv preprint arXiv:1502.02727*, 2015.
- [62] A. Lenz and N. Polyanski, "Optimal codes correcting a burst of deletions of variable length," in *2020 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2020, pp. 757–762.

- [63] A. Lenz, P. H. Siegel, A. Wachter-Zeh, and E. Yaakobi, "Anchor-based correction of substitutions in indexed sets," in *2019 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2019, pp. 757–761.
- [64] —, "Coding over sets for DNA storage," *IEEE Transactions on Information Theory*, vol. 66, no. 4, pp. 2331–2351, 2019.
- [65] —, "An upper bound on the capacity of the DNA storage channel," in *2019 IEEE Information Theory Workshop (ITW)*. IEEE, 2019, pp. 1–5.
- [66] —, "Achieving the capacity of the DNA storage channel," in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 8846–8850.
- [67] A. Lenz, A. Wachter-Zeh, and E. Yaakobi, "Duplication-correcting codes," *Designs, Codes and Cryptography*, vol. 87, pp. 277–298, 2019.
- [68] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," in *Soviet Physics Doklady*, vol. 10, no. 8. Soviet Union, 1966, pp. 707–710.
- [69] —, "Bounds for deletion/insertion correcting codes," in *Proceedings IEEE International Symposium on Information Theory*. IEEE, 2002, p. 370.
- [70] S. Liu and C. Xing, "Bounds and constructions for insertion and deletion codes," *IEEE Transactions on Information Theory*, 2022.
- [71] H. Lou, M. Schwartz, J. Bruck, and F. Farnoud, "Evolution of k -mer frequencies and entropy in duplication and substitution mutation systems," *IEEE Transactions on Information Theory*, vol. 66, no. 5, pp. 3171–3186, May 2020.
- [72] H. Mahdavi and A. Vardy, "Asymptotically optimal sticky-insertion-correcting codes with efficient encoding and decoding," in *2017 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2017, pp. 2683–2687.
- [73] A. Makur, "Coding theorems for noisy permutation channels," *IEEE Transactions on Information Theory*, vol. 66, no. 11, pp. 6723–6748, 2020.
- [74] H. Mercier, V. K. Bhargava, and V. Tarokh, "A survey of error-correcting codes for channels with symbol synchronization errors," *IEEE Communications Surveys & Tutorials*, vol. 12, no. 1, pp. 87–96, 2010.
- [75] M. Mitzenmacher, "A survey of results for deletion channels and related synchronization channels," 2009.
- [76] L. Organick, S. D. Ang, Y.-J. Chen, R. Lopez, S. Yekhanin, K. Makarychev, M. Z. Racz, G. Kamath, P. Gopalan, B. Nguyen *et al.*, "Scaling up DNA data storage and random access retrieval," *BioRxiv*, p. 114553, 2017.
- [77] A. Orlitsky, "Interactive communication of balanced distributions and of correlated files," *SIAM Journal on Discrete Mathematics*, vol. 6, no. 4, pp. 548–564, 1993.
- [78] A. N. Ravi, A. Vahid, and I. Shomorony, "Capacity of the torn paper channel with lost pieces," in *2021 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2021, pp. 1937–1942.
- [79] N. Raviv, M. Schwartz, and E. Yaakobi, "Rank-modulation codes for DNA storage with shotgun sequencing," *IEEE Transactions on Information Theory*, vol. 65, no. 1, pp. 50–64, 2018.
- [80] D. Sankoff and J. B. Kruskal, "Time warps, string edits, and macromolecules: the theory and practice of sequence comparison," *Reading: Addison-Wesley Publication*, 1983.
- [81] C. Schoeny, A. Wachter-Zeh, R. Gabrys, and E. Yaakobi, "Codes correcting a burst of deletions or insertions," *IEEE Transactions on Information Theory*, vol. 63, no. 4, pp. 1971–1985, 2017.
- [82] L. J. Schulman and D. Zuckerman, "Asymptotically good codes correcting insertions, deletions, and transpositions," *IEEE Transactions on Information Theory*, vol. 45, no. 7, pp. 2552–2557, 1999.
- [83] F. Sellers, "Bit loss and gain correction code," *IRE Transactions on Information theory*, vol. 8, no. 1, pp. 35–38, 1962.
- [84] T. Shinkar, E. Yaakobi, A. Lenz, and A. Wachter-Zeh, "Clustering-correcting codes," *IEEE Transactions on Information Theory*, vol. 68, no. 3, pp. 1560–1580, 2021.
- [85] S. L. Shipman, J. Nivala, J. D. Macklis, and G. M. Church, "CRISPR-Cas encoding of digital movie into the genomes of a population of living bacteria," *Nature*, vol. 547, pp. 345–349, Jul. 2017.
- [86] I. Shomorony and R. Heckel, "DNA-based storage: Models and fundamental limits," *IEEE Transactions on Information Theory*, vol. 67, no. 6, pp. 3675–3689, 2021.
- [87] I. Shomorony, R. Heckel *et al.*, "Information-theoretic foundations of DNA data storage," *Foundations and Trends® in Communications and Information Theory*, vol. 19, no. 1, pp. 1–106, 2022.
- [88] I. Shomorony and A. Vahid, "Torn-paper coding," *IEEE Transactions on Information Theory*, vol. 67, no. 12, pp. 7904–7913, 2021.
- [89] J. Sima and J. Bruck, "On optimal k -deletion correcting codes," *IEEE Transactions on Information Theory*, vol. 67, no. 6, pp. 3360–3375, 2020.
- [90] J. Sima, R. Gabrys, and J. Bruck, "Optimal systematic t -deletion correcting codes," in *2020 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2020, pp. 769–774.
- [91] J. Sima, N. Raviv, and J. Bruck, "Two deletion correcting codes from indicator vectors," *IEEE Transactions on Information Theory*, vol. 66, no. 4, pp. 2375–2391, 2019.
- [92] —, "Robust indexing-optimal codes for DNA storage," in *2020 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2020, pp. 717–722.
- [93] —, "On coding over sliced information," *IEEE Transactions on Information Theory*, vol. 67, no. 5, pp. 2793–2807, 2021.
- [94] N. J. Sloane, "On single-deletion-correcting codes," *Codes and designs*, vol. 10, pp. 273–291, 2002.
- [95] W. Song, K. Cai, and K. A. S. Immink, "Sequence-subset distance and coding for error control in DNA-based data storage," *IEEE Transactions on Information Theory*, vol. 66, no. 10, pp. 6048–6065, 2020.
- [96] W. Song, N. Polyanskii, K. Cai, and X. He, "On multiple-deletion multiple-substitution correcting codes," in *2021 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2021, pp. 2655–2660.
- [97] Y. Sun, Y. Zhang, and G. Ge, "Improved constructions of permutation and multi-permutation codes correcting a burst of stable deletions," *IEEE Transactions on Information Theory*, 2023.
- [98] S. Tabatabaei Yazdi, Y. Yuan, J. Ma, H. Zhao, and O. Milenkovic, "A rewritable, random-access DNA-based storage system," *Scientific Reports*, vol. 5, no. 1, pp. 1–10, 2015.
- [99] I. Tal, H. D. Pfister, A. Fazeli, and A. Vardy, "Polar codes for the deletion channel: Weak and strong polarization," *IEEE Transactions on Information Theory*, vol. 68, no. 4, pp. 2239–2265, 2021.
- [100] L. G. Tallini, N. Alqwaify, and B. Bose, "Deletions and insertions of the symbol '0' and asymmetric/unidirectional error control codes for the l metric," *IEEE Transactions on Information Theory*, vol. 69, no. 1, pp. 86–106, 2022.
- [101] J. Tang and Y. Polyanskiy, "Capacity of noisy permutation channels," *IEEE Transactions on Information Theory*, 2023.
- [102] Y. Tang and F. Farnoud, "Error-correcting codes for noisy duplication channels," *IEEE Transactions on Information Theory*, vol. 67, no. 6, pp. 3452–3464, Jun. 2021.

- [103] —, “Error-correcting codes for short tandem duplication and edit errors,” *IEEE Transactions on Information Theory*, vol. 68, no. 2, pp. 871–880, Feb. 2021.
- [104] Y. Tang, S. Wang, H. Lou, R. Gabrys, and F. Farnoud, “Low-redundancy codes for correcting multiple short-duplication and edit errors,” *IEEE Transactions on Information Theory*, 2023, accepted for publication.
- [105] Y. Tang, Y. Yehezkeally, M. Schwartz, and F. Farnoud, “Single-error detection and correction for duplication and substitution channels,” *IEEE Transactions on Information Theory*, vol. 66, no. 11, pp. 6908–6919, Nov. 2020.
- [106] G. Tenengolts, “Nonbinary codes, correcting single deletion or insertion (corresp.),” *IEEE Transactions on Information Theory*, vol. 30, no. 5, pp. 766–769, 1984.
- [107] J. Ullman, “On the capabilities of codes to correct synchronization errors,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 95–105, 1967.
- [108] R. R. Varshamov and G. M. Tenengolts, “Codes which correct single asymmetric errors,” *Automation and Remote Control*, vol. 26, no. 2, pp. 286–290, 1965.
- [109] R. Venkataramanan, S. Tatikonda, and K. Ramchandran, “Achievable rates for channels with deletions and insertions,” *IEEE Transactions on Information Theory*, vol. 59, no. 11, pp. 6990–7013, 2013.
- [110] J. M. Walsh and S. Weber, “Capacity region of the permutation channel,” in *2008 46th Annual Allerton Conference on Communication, Control, and Computing*. IEEE, 2008, pp. 646–652.
- [111] S. Wang, Y. Tang, R. Gabrys, and F. Farnoud, “Permutation codes for correcting a burst of at most t deletions,” in *2022 58th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*. IEEE, 2022, pp. 1–6.
- [112] S. Wang, V. K. Vu, and V. Y. Tan, “Codes for correcting t limited-magnitude sticky deletions,” *arXiv preprint arXiv:2302.02754*, 2023.
- [113] H. Wei and M. Schwartz, “Improved coding over sets for DNA-based data storage,” *IEEE Transactions on Information Theory*, vol. 68, no. 1, pp. 118–129, 2021.
- [114] N. Weinberger and N. Merhav, “The DNA storage channel: Capacity and error probability bounds,” *IEEE Transactions on Information Theory*, vol. 68, no. 9, pp. 5657–5700, 2022.
- [115] Wikipedia, “Edit distance,” https://en.wikipedia.org/wiki/Edit_distance.
- [116] P. C. Wong, K.-k. Wong, and H. Foote, “Organic data memory using the dna approach,” *Communications of the ACM*, vol. 46, no. 1, pp. 95–98, 2003.
- [117] S. Yazdi, R. Gabrys, and O. Milenkovic, “Portable and error-free DNA-based data storage,” *Scientific Reports*, vol. 7, no. 1, pp. 1–6, 2017.
- [118] S. H. T. Yazdi, H. M. Kiah, E. Garcia-Ruiz, J. Ma, H. Zhao, and O. Milenkovic, “DNA-based storage: Trends and methods,” *IEEE Transactions on Molecular, Biological and Multi-Scale Communications*, vol. 1, no. 3, pp. 230–248, 2015.
- [119] Y. Yehezkeally, D. Bar-Lev, S. Marcovich, and E. Yaakobi, “Generalized unique reconstruction from substrings,” *IEEE Transactions on Information Theory*, 2023.
- [120] Y. Yehezkeally and M. Schwartz, “Reconstruction codes for DNA sequences with uniform tandem-duplication errors,” *IEEE Transactions on Information Theory*, vol. 66, no. 5, pp. 2658–2668, May 2020.
- [121] —, “Uncertainty and reconstruction with list-decoding from uniform-tandem-duplication noise,” *IEEE Transactions on Information Theory*, vol. 67, no. 7, pp. 4276–4287, Jul. 2021.
- [122] S. S. Yim, R. M. McBee, A. M. Song, Y. Huang, R. U. Sheth, and H. H. Wang, “Robust direct digital-to-biological data storage in living cells,” *Nature Chemical Biology*, vol. 17, no. 3, pp. 246–253, 2021.
- [123] M. Zeraatpisheh, M. Esmaeili, and T. A. Gulliver, “Construction of tandem duplication correcting codes,” *IET Communications*, vol. 13, no. 15, pp. 2217–2225, 2019.
- [124] —, “Construction of duplication correcting codes,” *IEEE Access*, vol. 8, pp. 96 150–96 161, 2020.

Jin Sima is a postdoctoral researcher in the Department of electrical and computer engineering at University of Illinois Urbana-Champaign. He received a B.Eng. and a M.Sc. in electronic engineering from Tsinghua University, China, in 2013 and 2016 respectively, and a Ph.D in electrical engineering from California Institute of Technology (Caltech) in 2022. His research interests include information and coding theory, machine learning, and theory of computation. He is a recipient of the 2019 IEEE Jack Keil Wolf ISIT Student Paper Award, the 2020-2021 IEEE Communication Society Data Storage Best Paper Award, the 2022 Caltech Charles Wilts Prize for best doctoral thesis, and the 2023 Thomas M. Cover Dissertation Award.

Netanel Raviv (Senior Member, IEEE) is an Assistant Professor with the Department of Computer Science and Engineering, Washington University in St. Louis, St. Louis, MO, USA. He received a B.Sc. in mathematics and computer science, an M.Sc. and Ph.D. degrees in computer science from the Technion-Israel Institute of Technology, Israel, in 2010, 2013, and 2017, respectively. He His research interests include applications of coding techniques to privacy, distributed computations, and machine learning. He was an awardee of the IBM Ph.D. fellowship for the academic year of 2015–2016, the first prize in the Feder family competition for best student work in communication technology in 2017, and the Lester-Deutsche Postdoctoral Fellowship.

Moshe Schwartz is a professor at McMaster University. His research interests include algebraic coding, combinatorial structures, and digital sequences. Dr. Schwartz received the B.A.(summa cum laude), M.Sc., and Ph.D. degrees from the Technion in 1997, 1998, and 2004 respectively. He was a Fulbright post-doctoral researcher at UCSD and Caltech, and then a professor at Ben-Gurion University of the Negev. Dr. Schwartz received the 2009 IEEE Communications Society Best Paper Award in Signal Processing and Coding for Data Storage, and the 2020 NVMW Persistent Impact Prize. He has been serving as an Associate Editor and Area Editor for the IEEE Transactions on Information Theory since 2014, and as an Editorial Board Member for the Journal of Combinatorial Theory Series A since 2021.

Jehoshua Bruck is the Gordon and Betty Moore Professor of computation and neural systems and electrical engineering at the California Institute of Technology (Caltech). His current research interests include information theory and systems and the theory of computation in nature.

Dr. Bruck received his degrees in electrical engineering: B.Sc. (1982) and M.Sc. (1985) from the Technion and Ph.D. (1989) from Stanford University. His industrial experiences include working for IBM Research, cofounding and serving as Chairman of: Rainfinity (acquired by EMC), XtremIO (acquired by EMC), and of MemVerge. He is a recipient of the Feynman Prize for Excellence in Teaching, the Sloan Research Fellowship, the National Science Foundation Young Investigator Award, the IBM Outstanding Innovation Award and the IBM Outstanding Technical Achievement Award.